ФГБОУ ВО «Пензенский государственный университет»
Факультет вычислительной техники
Кафедра «Вычислительная техника»

ОТЧЕТ

по дисциплине «Логика и ОА»
лабораторная работа №6 на тему:
«Унарные и бинарные операции над графами»

Выполнил студенты группы 25ВВВ3:
Кустов И. С
Никитин К. А
Приняли:
к.т.н., доцент Юрова О. В.
к.т.н., доцент Деев М. В.

Пенза 2026

Тема: «Унарные и бинарные операции над графами»

Цель работы: изучить основные унарные и бинарные операции над графами: отождествление вершин, стягивание ребра, расщепление вершины, объединение, пересечение, декартово произведение и кольцевую сумму. Освоить матричную и списочную формы представления графов. Научиться генерировать случайные графы с помощью датчика случайных чисел, преобразовывать матрицу смежности в списки смежности и выполнять над графами заданные операции с выводом результата на экран.

Лабораторное задание:

Задание 1:

1.Сгенерируйте (используя генератор случайных чисел) две матрицы M1, M2 смежности неориентированных помеченных графов G1, G2. Выведите сгенерированные матрицы на экран.

2.Для указанных графов преобразуйте представление матриц смежности в списки смежности. Выведите полученные списки на экран.

Задание 2:

1.Для матричной формы представления графов выполните операцию:

а) отождествления вершин

б) стягивания ребра

в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры. Результат выполнения операции выведите на экран.

2. Для представления графов в виде списков смежности выполните операцию:

а) отождествления вершин

б) стягивания ребра

в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры. Результат выполнения операции выведите на экран.

Задание 3:

1.Для матричной формы представления графов выполните операцию:

а) объединения G = G1 G2

б) пересечения G = G1 G2

в) кольцевой суммы G = G1 ⊕ G2

Результат выполнения операции выведите на экран.

Задание 4:

1.Для матричной формы представления графов выполните операцию декартова произведения графов G = G1 X G2.

Результат выполнения операции выведите на экран.

Пояснительный текст:

Задание 1:

Часть 1. Генерация и вывод матриц смежности

1.1. Ввод размеров графов

В начале main() программа запрашивает у пользователя количество вершин для каждого графа:

```
printf("Введите количество вершин для графа G1: ");
if (scanf("%d", &n1) != 1 || n1 < 1) n1 = 4; // по умолчанию 4
printf("Введите количество вершин для графа G2: ");
if (scanf("%d", &n2) != 1 || n2 < 1) n2 = 4;
```

Если пользователь введёт некорректное значение (не число или меньше 1), устанавливается значение по умолчанию — 4.

1.2. Создание матриц

Для каждого графа выделяется динамическая память под матрицу размера n × n с помощью функции createMatrix:

```
int** createMatrix(int n) {
    int** matrix = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        matrix[i] = (int*)malloc(n * sizeof(int));
    }
    return matrix;
```

```
}
```

Сначала выделяется массив указателей на строки, затем для каждой строки — массив из n целых чисел.

1.3. Генерация случайного графа

Функция generateGraph заполняет матрицу случайными значениями 0 или 1, обеспечивая свойства неориентированного графа без петель:

```
void generateGraph(int** matrix, int n) {
    for (int i = 0; i < n; i++) {
        matrix[i][i] = 0; // петли отсутствуют
        for (int j = i + 1; j < n; j++) {
            int val = rand() % 2; // 0 или 1
            matrix[i][j] = val;
            matrix[j][i] = val; // симметрия для неориентированного графа
        }
    }
}
```

- Главная диагональ обнуляется (петель нет).
- Верхний треугольник (i < j) заполняется случайными 0/1.
- Нижний треугольник получает те же значения, что и верхний, чтобы матрица была симметричной (граф неориентированный).

1.4. Вывод матриц на экран

Функция printMatrix выводит матрицу в удобочитаемом виде с нумерацией строк и столбцов:

```
void printMatrix(int** matrix, int n, const char* title) {
    printf("\n--- %s (размер: %d x %d) ---\n", title, n, n);
    // Вывод заголовка столбцов
    printf("    ");
    for (int j = 0; j < n; j++) printf("%3d", j + 1);
    printf("\n    ");
    for (int j = 0; j < n; j++) printf("---");
```

```
    printf("\n");
    // Вывод строк
    for (int i = 0; i < n; i++) {
        printf("%3d |", i + 1);
        for (int j = 0; j < n; j++) {
            printf("%3d", matrix[i][j]);
        }
        printf("\n");
    }
}
```

Выводятся заголовки 1, 2, ..., n, затем каждая строка с номером вершины и значениями 0/1.

1.5. Вызов в main

В main() после ввода размеров и создания матриц выполняются:

```
generateGraph(G1, n1);
generateGraph(G2, n2);
printMatrix(G1, n1, "Исходный граф G1");
printMatrix(G2, n2, "Исходный граф G2");
```

Таким образом, сразу при запуске программы выводятся две матрицы смежности — это и есть выполнение первой части Задания 1.

Часть 2. Преобразование в списки смежности и их вывод

Эта часть выполняется по выбору пользователя в меню (пункт 2).

2.1. Структура списка смежности

Для представления списков используется динамическая структура:

```
typedef struct Node {
    int vertex;          // номер смежной вершины
    struct Node* next;   // указатель на следующий узел
} Node;
```

Каждая вершина графа имеет свой односвязный список смежных вершин.

2.2. Добавление ребра в список

Функция addEdgeToList добавляет новый узел в начало списка для вершины u:

```
void addEdgeToList(Node** adjList, int u, int v) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->vertex = v;
    newNode->next = adjList[u];
    adjList[u] = newNode;
}
```

Вставка в начало выбрана для простоты и эффективности O(1).

2.3. Преобразование матрицы в списки

Функция matrixToAdjList создаёт массив списков и заполняет его на основе матрицы смежности:

```
Node** matrixToAdjList(int** matrix, int n) {
    Node** adjList = (Node**)malloc(n * sizeof(Node*));
    for (int i = 0; i < n; i++) {
        adjList[i] = NULL;
    }
    for (int i = 0; i < n; i++) {
        for (int j = n - 1; j >= 0; j--) { // обратный порядок
            if (matrix[i][j] == 1) {
                addEdgeToList(adjList, i, j);
            }
        }
    }
    return adjList;
}
```

- Для каждой строки i просматриваются столбцы j **в обратном порядке** (от n-1 до 0). Это нужно, чтобы после вставки в начало списка смежные вершины шли в **возрастающем порядке**.

- Если matrix[i][j] == 1, вершина j добавляется в список вершины i.

2.4. Вывод списков на экран

Функция printAdjList выводит для каждой вершины её список смежности:

```
void printAdjList(Node** adjList, int n) {
    printf("\n=== Списки смежности ===\n");
    for (int i = 0; i < n; i++) {
        printf("Вершина %d: ", i + 1);
        Node* curr = adjList[i];
        if (!curr) {
            printf("[нет связей]");
        }
        while (curr != NULL) {
            printf("-> %d ", curr->vertex + 1);
            curr = curr->next;
        }
        printf("\n");
    }
}
```

Выводятся номера вершин (начиная с 1) и цепочка смежных с ними вершин.

2.5. Вызов в меню

В главном меню пункт 2 обрабатывается так:

```
case 2: {
    Node** list1 = matrixToAdjList(G1, n1);
    printf("\nСписки смежности для G1:");
    printAdjList(list1, n1);
    freeAdjList(list1, n1); // освобождение памяти

    Node** list2 = matrixToAdjList(G2, n2);
```

```
        printf("\nСписки смежности для G2:");
        printAdjList(list2, n2);
        freeAdjList(list2, n2);
        break;
    }
```

Пользователь выбирает пункт 2, и программа строит списки смежности для обоих графов, выводит их и освобождает память.

2.6. Освобождение памяти

Функция freeAdjList рекурсивно (или итеративно) освобождает все узлы списков и массив указателей:

```
    void freeAdjList(Node** adjList, int n) {
        if (!adjList) return;
        for (int i = 0; i < n; i++) {
            Node* curr = adjList[i];
            while (curr != NULL) {
                Node* tmp = curr;
                curr = curr->next;
                free(tmp);
            }
        }
        free(adjList);
    }
```

Задание 2:

а) Отождествление вершин. identification(matrix, n, u, v, out_n)

Схлопываем две вершины u и v в одну. Итоговый граф имеет на одну вершину меньше.

Как работает:

1. Новый размер: new_n = n - 1. Результат записывается в *out_n, чтобы вызывающая функция знала размер.

2. Гарантия порядка: если u > v, они меняются местами. Это нужно, чтобы вычёркивалась всегда вершина с большим индексом.

3. Копирование с вычёркиванием: проходим по всем i и j исходной матрицы, пропуская строку и столбец v (вершина v удаляется). Для каждой пары (i, j):

- Если i == u или j == u — вершина u объединяет окружения обеих вершин:

```
int val_from_u = matrix[i][j];
int val_from_v = (i == u) ? matrix[v][j] : matrix[i][v];
res[ni][nj] = val_from_u || val_from_v;
```

Логическое ИЛИ (||) означает: ребро остаётся, если оно было хотя бы у одной из вершин (u или v). Это и есть объединение окружений Q = Q1 ∪ Q2 по определению из теории.

- Иначе значение копируется как есть.

4. Убираем петлю: res[u][u] = 0, так как после объединения двух вершин на диагонали могла бы возникнуть петля, но в задании её быть не должно.

Вершина v вычёркивается, а её связи объединяются со связями u через логическое ИЛИ. Размер графа уменьшается на 1.

б) Стягивание ребра. contractEdge(matrix, n, u, v, out_n)

Суть: То же отождествление, но только если между u и v действительно есть ребро.

Как работает:

1. Проверка ребра: первым делом проверяем if (matrix[u][v] == 0). Если ребра нет — выводим сообщение об ошибке и возвращаем NULL. Стягивать нечего.

2. Если ребро есть: вызываем identification(matrix, n, u, v, out_n). То есть вся работа уже реализована в предыдущей функции.

Стягивание — это отождествление смежных вершин. Сначала проверяем, что ребро существует (matrix[u][v] == 1), иначе операция невозможна. Если ребро есть — вызываем identification.

Пример: если u=1, v=2, и matrix[0][1] == 1 — стягиваем. Если matrix[0][1] == 0 — выводим ошибку.

в) Расщепление вершины. splitVertex(matrix, n, u, out_n)

Суть: Раздвоение вершины на две. Операция, обратная стягиванию. Размер графа увеличивается на 1.

Как работает:

1. Новый размер: new_n = n + 1. В *out_n записывается новое число вершин.

2. Копирование базовой матрицы: исходная матрица n × n копируется в левый верхний угол новой (n+1) × (n+1). Добавленная строка и столбец обнуляются.

3. Соединяем старую и новую вершины: добавляем ребро между u и новой вершиной с номером n:

res[u][n] = 1;

res[n][u] = 1;

4. Распределение соседей: самый интересный момент. Проходим по всем соседям вершины u (кроме неё самой) и каждого второго отдаём новой вершине:

```
int count = 0;
for (int i = 0; i < n; i++) {
    if (i != u && matrix[u][i] == 1) {
        count++;
        if (count % 2 == 0) {
            // передаём ребро новой вершине
            res[u][i] = 0;
            res[i][u] = 0;
            res[n][i] = 1;
            res[i][n] = 1;
        }
    }
```

}

Логика: чётные соседи (2-й, 4-й, 6-й…) отходят новой вершине, нечётные (1-й, 3-й, 5-й…) остаются у старой. Это позволяет разделить связи примерно пополам.

Расщепление — операция, обратная стягиванию. Добавляем новую вершину, соединяем её ребром со старой, а соседей делим: чётные отдаём новой вершине, нечётные оставляем старой. Размер графа увеличивается на 1.

Как это выглядит в меню

В главном меню (пункты 3, 4, 5) для каждой операции:

1. Программа запрашивает номера вершин (для отождествления и стягивания — две, для расщепления — одну).

2. Проверяет корректность: номера в диапазоне 1..n1, для первых двух u != v.

3. Вызывает соответствующую функцию.

4. Выводит результат — **и как матрицу смежности, и как списки смежности** (для наглядности).

Пример для пункта 3:

```
int res_n;
int** res = identification(G1, n1, u - 1, v - 1, &res_n);
printMatrix(res, res_n, "Результат отождествления");

Node** list = matrixToAdjList(res, res_n);
printAdjList(list, res_n);
freeAdjList(list, res_n);
freeMatrix(res, res_n);
```

Задание 3:

Вспомогательная функция getEdge(m, n, i, j) — Безопасное чтение

Зачем нужна: при бинарных операциях графы могут иметь разные размеры. Если один граф больше другого, при попытке прочитать элемент за границей меньшего графа программа «упадёт» с ошибкой доступа к памяти.

Как работает:

```
int getEdge(int** m, int n, int i, int j) {
    if (i < n && j < n) {
        return m[i][j];
    }
    return 0;  // если за границей — считаем, что ребра нет
}
```

Если индексы i или j выходят за размер n, функция возвращает 0 — то есть «ребра нет, вершина изолирована». Это защищает от вылета и логически корректно: у меньшего графа просто нет таких вершин.

getEdge — безопасное чтение. Если графы разного размера, при попытке прочитать элемент за границей меньшего графа возвращается 0 (изолированная вершина), защищая от вылета программы.

а) Объединение (наложение) graphUnion(m1, n1, m2, n2, out_n)

Формула: m1 || m2 (логическое ИЛИ).

Смысл: Ребро появляется в результате, если оно было в первом графе ИЛИ во втором (или в обоих сразу).

Как работает:

1. Размер результата: берётся максимум из n1 и n2:

int n = (n1 > n2) ? n1 : n2;

Это логично: если один граф больше, в нём есть вершины, которых нет в другом, и они должны сохраниться.

2. Обход всех пар (i, j): для каждой позиции результирующей матрицы:

res[i][j] = getEdge(m1, n1, i, j) || getEdge(m2, n2, i, j);

- Если ребро есть хотя бы в одном графе → 1.
- Если ребра нет нигде → 0.

Пример: пусть G1 = 3×3, G2 = 3×3, и в G1 есть ребро (1,2), а в G2 — ребро (2,3). После объединения будут оба ребра: (1,2) и (2,3).

Объединение — логическое ИЛИ. Размер берём по максимуму, чтобы не потерять вершины большего графа. Ребро остаётся, если оно было хотя бы в одном из графов.

б) Пересечение. graphIntersection(m1, n1, m2, n2, out_n)

Формула: m1 && m2 (логическое И).

Смысл: Ребро остаётся, только если оно было в обоих графах одновременно.

Как работает:

1. Размер результата: берётся минимум из n1 и n2:

int n = (n1 < n2) ? n1 : n2;

Это логично: если в одном графе нет какой-то вершины, то ребра с её участием не могут быть «общими».

2. Обход всех пар (i, j):

res[i][j] = m1[i][j] && m2[i][j];

- Если ребро есть в обоих графах → 1.
- Иначе → 0.

Здесь getEdge не нужен, потому что мы уже ограничили размеры до min(n1, n2) — обе матрицы точно содержат нужные элементы.

Пример: G1 имеет рёбра (1,2) и (1,3), G2 — (1,2) и (2,3). Пересечение даст только (1,2).

Пересечение — логическое И. Размер берём по минимуму, потому что общие рёбра могут быть только у общих вершин. Ребро остаётся, только если оно было в обоих графах.

в) Кольцевая сумма. graphRingSum(m1, n1, m2, n2, out_n)

Формула: m1 ^ m2 (побитовый XOR, исключающее ИЛИ).

Смысл: Ребро есть, если оно присутствует только в одном из графов. Если ребро было в обоих — оно исчезает (1 ^ 1 = 0).

Как работает:

1. Размер результата: берётся максимум из n1 и n2:

int n = (n1 > n2) ? n1 : n2;

Как и в объединении — чтобы не потерять вершины большего графа.

2. Обход всех пар (i, j):

res[i][j] = getEdge(m1, n1, i, j) ^ getEdge(m2, n2, i, j);

Таблица истинности XOR:

m1	m2	результат
0	0	0
0	1	1
1	0	1
1	1	0

Пример: G1 имеет (1,2) и (1,3), G2 — (1,2) и (2,3). После кольцевой суммы:

- (1,2): 1 ^ 1 = 0 — исчезает.
- (1,3): 1 ^ 0 = 1 — остаётся.
- (2,3): 0 ^ 1 = 1 — остаётся.

Результат: рёбра (1,3) и (2,3).

Кольцевая сумма — побитовый XOR. Ребро есть, если оно присутствует только в одном из графов. Если ребро было в обоих — оно исчезает (1 ^ 1 = 0). Размер берём по максимуму, как в объединении.

Как это выглядит в меню

В главном меню (пункты 6, 7, 8) для каждой операции:

```
case 6: {
    int res_n;
    int** res = graphUnion(G1, n1, G2, n2, &res_n);
    printMatrix(res, res_n, "Результат объединения G1 U G2");
    freeMatrix(res, res_n);
    break;
}
```

Номера вершин вводить не нужно — операция применяется ко всем вершинам сразу. Результат выводится только в виде матрицы (списки

смежности для бинарных операций в меню не показываются, но их можно получить через matrixToAdjList, если нужно).

Задание 4:

Суть операции

Если G1 имеет n1 вершин, а G2 — n2 вершин, то в декартовом произведении G = G1 × G2 будет n1 * n2 вершин. Каждая вершина — это пара (u, v), где u — вершина из G1, v — вершина из G2.

Условие смежности: две вершины (u1, v1) и (u2, v2) соединены ребром тогда и только тогда, когда выполнено хотя бы одно из двух условий:

1. u1 == u2 (первая компонента совпадает) и v1 смежна с v2 в G2.

2. v1 == v2 (вторая компонента совпадает) и u1 смежна с u2 в G1.

Это похоже на ход ладьи по прямоугольной сетке: можно двигаться либо по горизонтали (меняя v при фиксированном u), либо по вертикали (меняя u при фиксированном v), но не по диагонали.

Реализация в коде

Функция cartesianProduct:

```
int** cartesianProduct(int** m1, int n1, int** m2, int n2, int* out_n) {
    int total_n = n1 * n2;
    *out_n = total_n;
    int** res = createMatrix(total_n);

    // обнуляем матрицу
    for (int i = 0; i < total_n; i++) {
        for (int j = 0; j < total_n; j++) {
            res[i][j] = 0;
        }
    }

    for (int u1 = 0; u1 < n1; u1++) {
        for (int v1 = 0; v1 < n2; v1++) {
```

```
            int row = u1 * n2 + v1;   // плоский индекс вершины (u1, v1)

            for (int u2 = 0; u2 < n1; u2++) {
                for (int v2 = 0; v2 < n2; v2++) {
                    int col = u2 * n2 + v2; // плоский индекс вершины (u2, v2)

                    if ((u1 == u2 && m2[v1][v2] == 1) ||
                        (v1 == v2 && m1[u1][u2] == 1)) {
                        res[row][col] = 1;
                    }
                }
            }
        }
    }
    return res;
}
```

Разбор по шагам:

1. **Количество вершин:** total_n = n1 * n2. Это и есть размер результирующей матрицы.

2. **Плоский индекс:** каждой паре (u, v) ставится в соответствие число u * n2 + v. Это стандартный способ отображения двумерного массива в одномерный. Например, при n1=2, n2=3:

- (0,0) → 0
- (0,1) → 1
- (0,2) → 2
- (1,0) → 3
- (1,1) → 4
- (1,2) → 5

3. **Двойной цикл по всем парам вершин:** внешние два цикла перебирают первую вершину (u1, v1), внутренние два — вторую (u2, v2). Для каждой пары вычисляются row и col.

4. **Проверка условия смежности:**

if ((u1 == u2 && m2[v1][v2] == 1) || (v1 == v2 && m1[u1][u2] == 1))

- Первая часть: если u1 == u2 и в G2 есть ребро между v1 и v2, то ставим 1.
- Вторая часть: если v1 == v2 и в G1 есть ребро между u1 и u2, то ставим 1.
- В остальных случаях остаётся 0.

5. **Возврат результата:** матрица res размера total_n × total_n передаётся в main, где выводится на экран.

Пример работы

Пусть G1 — граф с 2 вершинами и одним ребром (0–1). G2 — граф с 3 вершинами и рёбрами (0–1) и (1–2). Тогда G1 × G2 будет иметь 2 * 3 = 6 вершин:

(0,0), (0,1), (0,2), (1,0), (1,1), (1,2)

Рёбра:

- Из-за G2 (фиксируем u): (0,0)–(0,1), (0,1)–(0,2), (1,0)–(1,1), (1,1)–(1,2).
- Из-за G1 (фиксируем v): (0,0)–(1,0), (0,1)–(1,1), (0,2)–(1,2).

Итоговый граф выглядит как решётка 2×3.

Вызов в меню

В главном меню пункт 9:

```
case 9: {
    int res_n;
    int** res = cartesianProduct(G1, n1, G2, n2, &res_n);
    printMatrix(res, res_n, "Результат декартова произведения G1 x G2");
    freeMatrix(res, res_n);
    break;
}
```

Номера вершин вводить не нужно — операция применяется ко всем вершинам автоматически. Результат выводится в виде матрицы смежности размера (n1*n2) × (n1*n2).

Декартово произведение G1 × G2 — это граф, вершинами которого являются все пары (u, v), где u ∈ G1, v ∈ G2. Всего вершин n1 * n2. Две вершины (u1, v1) и (u2, v2) смежны, если либо u1 = u2 и v1 смежна с v2 в G2, либо v1 = v2 и u1 смежна с u2 в G1. В коде это реализовано через четыре вложенных цикла: внешние два перебирают первую вершину, внутренние — вторую. Плоский индекс вычисляется как u * n2 + v. Условие проверяется логическим ИЛИ двух условий.

Параметр	Значение
Число вершин	n1 * n2
Размер матрицы	(n1*n2) × (n1*n2)
Условие ребра	(u1==u2 && m2[v1][v2]) || (v1==v2 && m1[u1][u2])
Сложность	O((n1*n2)^2)

Вывод:

В ходе выполнения лабораторной работы были изучены и программно реализованы основные унарные и бинарные операции над графами. Освоены две формы представления графов: матрица смежности и списки смежности, а также реализовано преобразование между ними.